

■ Relatório de Análise Crítica — Servidor Multithread

■ 1. Descrição Geral

O código implementa um servidor TCP multithread que aceita múltiplos clientes simultaneamente, utiliza mutexes, condições e semáforos para sincronização, e emprega uma fila de mensagens compartilhada entre threads produtoras e uma consumidora. A arquitetura é bem estruturada, mas há riscos de race conditions e possíveis deadlocks sob alta carga.

■ 2. Análise de Concorrência e Sincronização

Análise detalhada das principais seções críticas do código, com foco em estruturas ClientList, MessageQueue, broadcaster, threads de clientes e rotina de encerramento.

■ Estrutura ClientList

Potencial race condition ao enviar mensagens para sockets fechados. Recomenda-se adicionar flag de cliente ativo ou duplicar array dentro do lock para evitar inconsistências.

■ Estrutura MessageQueue

Possível deadlock durante shutdown, pois apenas um semáforo é postado. Sugere-se múltiplos sem_post() e checagem de shutdown dentro do lock.

■ Thread Broadcaster

Pode tentar enviar mensagens para sockets já fechados (EPIPE). Também há risco de vazamento de memória se msg não for liberada em falha.

■ Thread Client Handler

Possível race condition entre encerramento e recv(), e risco de double free caso cli seja liberado enquanto ainda é acessado.

■ Cleanup e Shutdown

Deadlock leve possível durante encerramento. Recomenda-se finalizar broadcaster antes dos clientes e garantir pthread_join nas threads.

Tipo	Local	Descrição	Gravidade	Sugestão
Race condition	broadcaster / clientlist	Envio para socket já fechado	Média	Validar send() e flag active
Race condition	msgqueue	Leitura de shutdown fora do lock	Leve	Mover checagem para dentro do lock
Deadlock	msgqueue_dequeue	Threads bloqueadas após shutdown	Alta	Usar múltiplos sem_post()
Dangling pointer	client_handler	free(cli) enquanto acessado	Alta	Usar flag de ativo antes de liberar
Leak	broadcaster	msg não liberada em falha	Leve	Garantir free(msg)

■ 3. Conclusão

O servidor é funcional e bem projetado, mas vulnerável a problemas de concorrência sob alta carga. Com ajustes nos semáforos, validação de sockets e controle de threads, o sistema pode se tornar robusto e seguro.